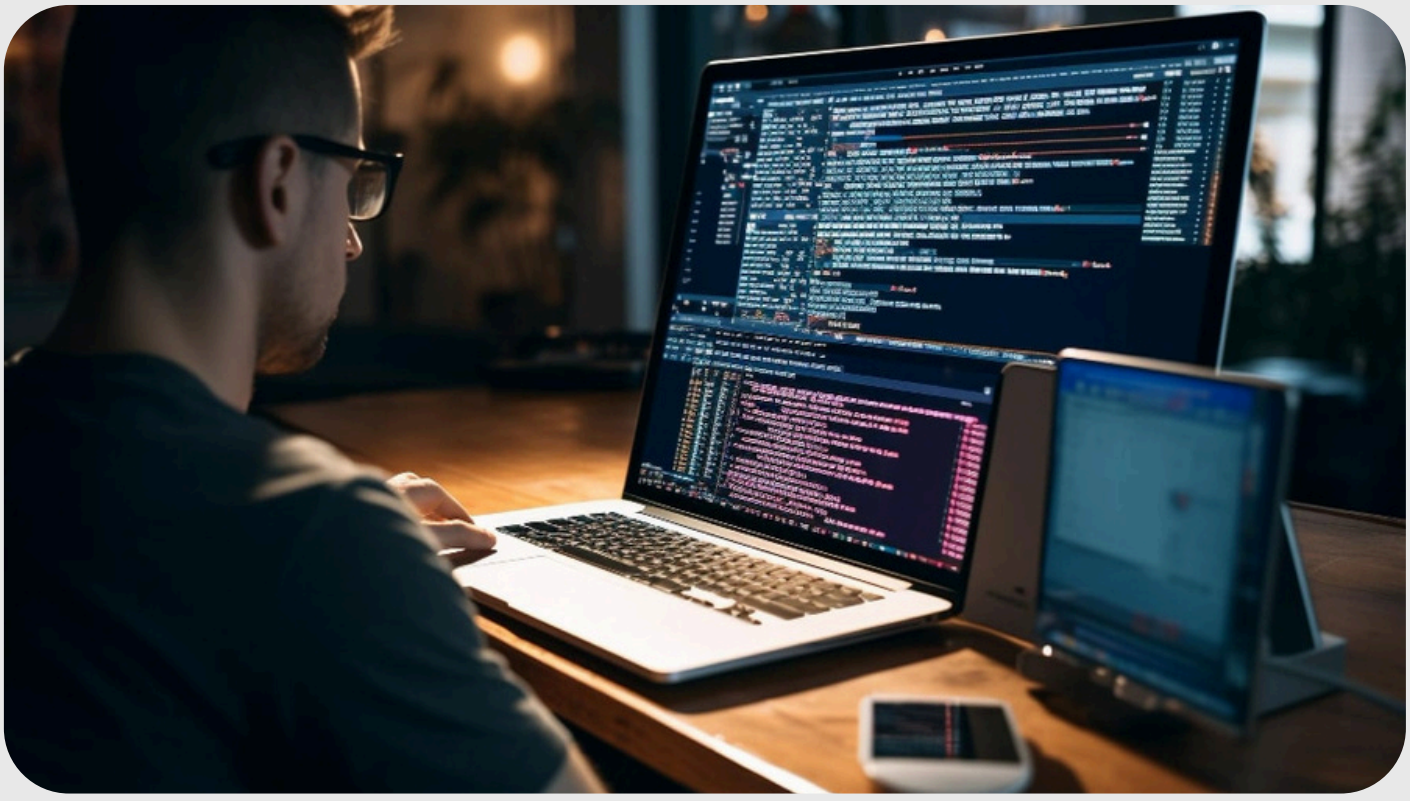
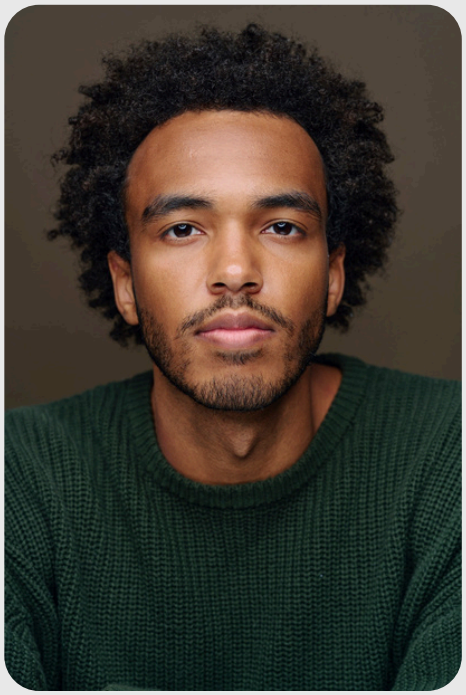
Master Your Coding Foundations



BASHIRI SMITH

Hey!



A LITTLE ABOUT ME

My name is Bashiri Smith.

In just under 12 months & without a college degree or any work experience, I received 2 offers for more than $115,000 for my first job as a software engineer.

Since then, I have gone on to earn over $265,000p/y as a software engineer and I have also founded my own tech startup. (Interlade)

Throughout my journey to learn how to code, I've found that quality education has been withheld behind unnecessary college degrees and overly-expensive coding bootcamps.

My mission is to make it easy for anyone to change their life with coding!

This guide is for aspiring coders who want to earn $100k+ as a software engineer in less than a year.

01

Foundational Programming Concepts

What is Coding/Programming?

Coding, or programming, is the process of creating instructions for computers to follow. It involves writing code in a programming language, which is a special language understood by computers. This code tells the computer what actions to perform, enabling it to execute various tasks, from simple calculations to controlling complex systems. Think of it as writing a recipe that the computer follows to complete a specific task or solve a problem.

What are Programming Languages?

Programming languages are like special codes that tell computers what to do. Think of them as the secret languages that computer programmers use to make games, apps, and websites. Just like we use English or Spanish to talk to each other, programmers use these languages to give instructions to computers.

There are lots of different programming languages, and they can be grouped into a few types:

1. High-level languages are the easiest for people to understand. They are a bit like talking in full sentences. Examples include Python, Java, C++, and JavaScript. Programmers use these for making all sorts of things, like video games, apps, and websites.
2. Low-level languages are more like the computer's own language. They're harder for people to understand but let the programmer control the computer very closely. Assembly language is an example. It's used for special jobs that need the programmer to manage exactly how the computer works.
3. Markup languages aren't exactly programming languages, but they help set up how things look on websites or organize information. HTML, which helps create web pages, is an example.

Each programming language has its own set of rules on how to write and organize code, kind of like how English has grammar and spelling rules. When programmers write code, they have to follow these rules so the computer can understand them.

Choosing which programming language to use depends on what the programmer wants to make. Some languages are better for certain tasks than others, like how some tools are better for certain jobs.

01

Foundational Programming Concepts

What is an IDE?

An IDE, or Integrated Development Environment, is a software application that provides comprehensive facilities to computer programmers for software development. Think of it as a toolbox that contains all the tools a programmer needs to write, test, and debug their code all in one place. An IDE typically includes:

- A code editor: This is where you write your code. The editor often highlights syntax and can auto-complete some of your code, making it easier to read and write.
- A compiler or interpreter: This tool translates your written code into a language that computers can understand, either by compiling the code into executable files or by interpreting the code directly.
- A debugger: This tool helps you find and fix errors in your code. It allows you to run your program step by step, inspect variables, and understand what your code is doing at each step.
- Build automation tools: These tools automate common development tasks, like compiling your code into an executable application or deploying your web application to a server.

Some IDEs are designed for a specific programming language, such as Eclipse or IntelliJ IDEA for Java, while others, like Visual Studio Code or Atom, can be used with a wide variety of languages through the use of plugins. IDEs can greatly improve a programmer's productivity by providing all the necessary tools in a single application, making it easier to write, test, and debug software.

VSCode industry standard among most modern Frontend/Backend/FullStack Engineers (Free Download Below):

https://code.visualstudio.com/download

01

Foundational Programming Concepts

Terminal

As a software engineer, using the terminal or command line is essential because it provides a powerful and direct way to interact with your computer's operating system and software. It allows you to perform tasks more efficiently and quickly than through a graphical user interface (GUI). With the command line, you can execute commands for compiling code, managing files, accessing databases, and controlling version systems like Git. It's also indispensable for automation, scripting, and working with servers or development environments that don't have a GUI. Learning to use the terminal can significantly enhance your productivity, troubleshooting skills, and understanding of how systems work under the hood.

Command Line Crash Course:
https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Understanding_client-side_tools/Command_line

01

Foundational Programming Concepts

- **Basic Syntax and Logic:** Familiarity with control structures, loops, functions, and data structures.
    - https://csx.codesmith.io/home

- Algorithms and Data Structures: Sorting, searching, stacks, queues, linked lists, trees, graphs, hash tables.
    - https://neetcode.io/courses/dsa-for-beginners

- Coding Challenges: Solve basic to intermediate-level coding problems like FizzBuzz, palindrome checks, and two-sum problems.
    - https://neetcode.io/practice

02

2 Best Youtube Channels & How to Use Them

FreeCodeCamp

What it's Good for:

- Extremely informational videos cover a wide range of topics and languages. Their "Course" videos are typically 1-4 hours long and go quite in depth on the subject.

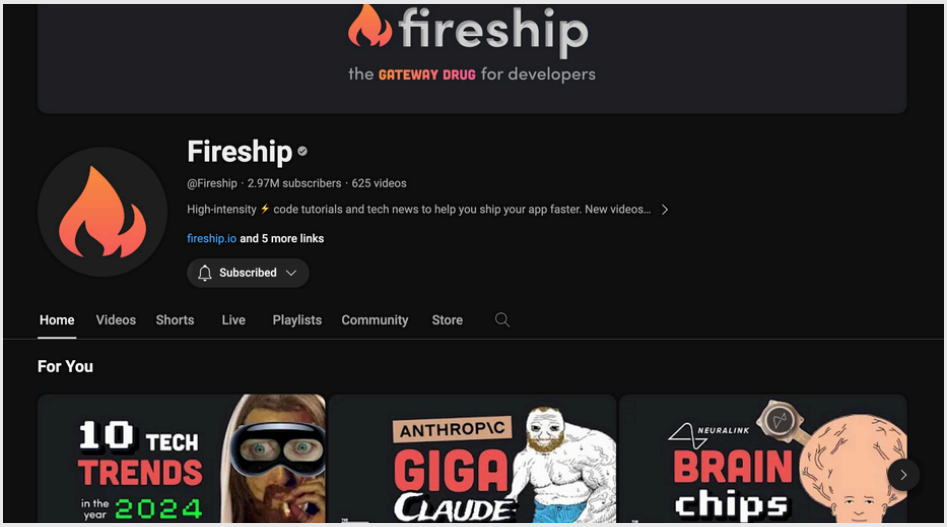
How to Use FreeCodeCamp:

- If you are able to stay disciplined enough to attentively watch through every video they have on a topic you want to learn, you wouldn't need anything else to learn to code. I've used their videos to learn while on the job!

03

2 Best Youtube Channels & How to Use Them

Fireship



What it's Good for:

- Great way to immerse into coding culture. You won't understand every video or topic, but by consistently watching these videos you will learn a lot about the industry.

How to Use Fireship:

- Use this channel as a substitute for your typical youtube entertainment. Doing so will rapidly increase your general knowledge on the coding industry and will improve your ability to chat about obscure topics in job interviews.

04

The Best Free Book & How to Use It

Programming Fundamentals, A Modular Structured Approach

What it's Good for:

- Great explainations of the basics of programming. It breaks down big programming ideas into small, easy-to-understand parts. The book talks about important programming topics like different kinds of data, how to control programs, organizing data, etc. This book was published in 2009 but the topics are still important to this day.

How to Use Programming Fundamentals:

- If you have no knowledge on coding at all I would recommend you watch a few youtube videos on "What coding is?". Once you have just a little understanding, **read this book cover to cover.**
- If you are already learning to code use this book as a **reference when you are confused on a basic concept.**

Book Summary

1. **Master the Basics of Programming**: Understand core concepts such as variables, data types, control structures (if statements, loops), functions, and error handling. Solidifying your foundation in these areas enables you to tackle more complex problems effectively.

2. **Develop Problem-Solving Skills**: Programming is essentially problem-solving. Learn how to break down complex problems into manageable parts, use algorithms and data structures efficiently, and apply logical thinking to devise solutions.

3. **Learn Object-Oriented Programming (OOP)**: OOP is a fundamental programming paradigm used in many high-level languages. Grasp the concepts of classes, objects, inheritance, encapsulation, and polymorphism. Understanding OOP principles is crucial for developing scalable and maintainable software.

4. **Understand Software Development Lifecycles (SDLC)**: Familiarize yourself with different stages of software development, including planning, analysis, design, implementation, testing, and maintenance. Knowledge of methodologies like Agile or Scrum is highly valuable in today's tech environment.

Book Summary Pt.2

5. **Practice Version Control**: Version control systems like Git are essential tools for software engineers. They allow you to track and manage changes to your codebase, collaborate with others, and contribute to open source projects.

6. **Develop Soft Skills**: Communication, teamwork, and problem-solving skills are as important as technical abilities. The ability to work effectively in a team, articulate ideas clearly, and adapt to change can significantly impact your career trajectory.

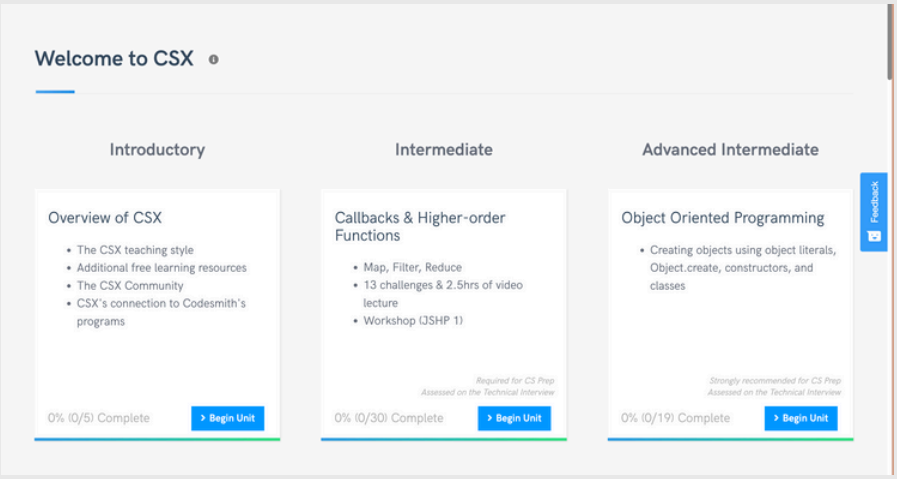
7. **Build a Portfolio and Contribute to Open Source**: Start personal projects and contribute to open-source projects to demonstrate your skills, learn from real-world codebases, and make your resume stand out. This practical experience is invaluable in job interviews.

8. **Stay Curious and Keep Learning**: The tech field evolves rapidly. Continuously learning new programming languages, technologies, and tools is crucial. Follow industry trends, attend workshops and conferences, and engage with the community through forums and social media.

06

2 Best Free Coursework & How to Use Them

CSX



What it's Good for:

- Best learning material for learning fundamental coding concepts. Uses javascript as the language you are practicing with, which is a great general language with tons of job opportunities.

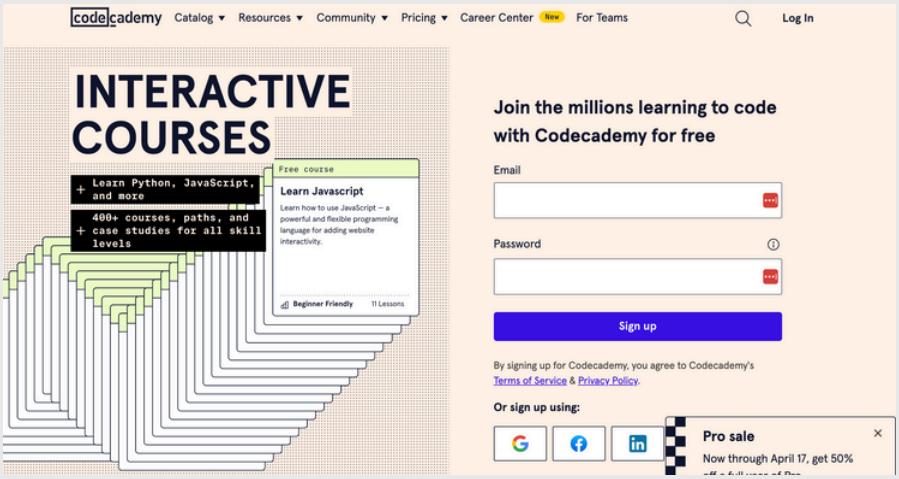
How to Use CSX:

- CSX was made by Codesmith, the $21,000 bootcamp I went to that helped me get a job. **Go through the entire course and complete every task.** Except the tasks that require you to schedule a meeting with the Codesmith team. If you have the money to pay for the bootcamp, go for it! If not use this free material to level up!

07

2 Best Free Coursework & How to Use Them

Codecademy



What it's Good for:

- Tons of full courses on many languages and industry topics. Has in order, step-by-step instructions of what you need to learn for many specific coding jobs.

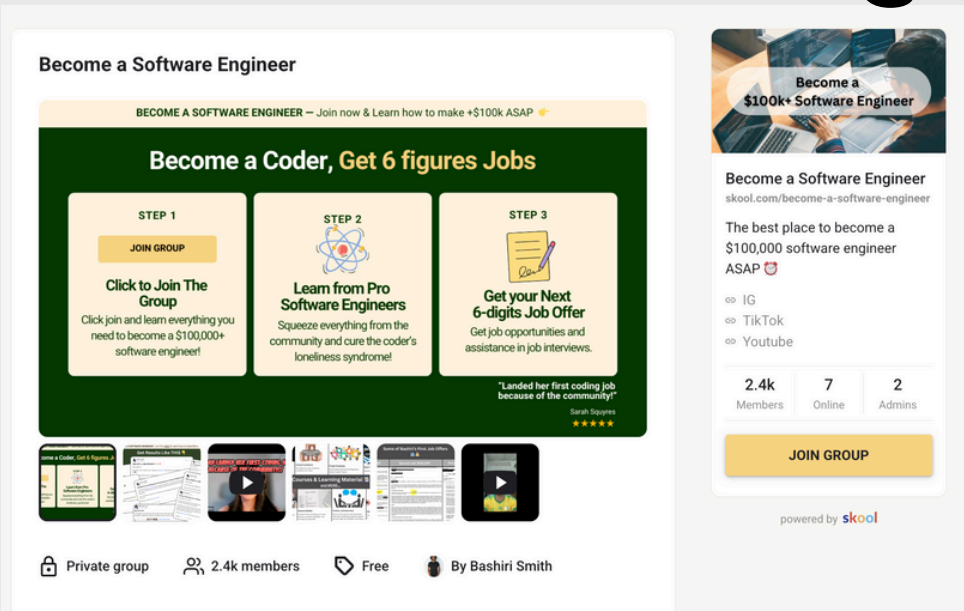
How to Use Codecademy:

- They have plenty of free material available and I would definitely recommend completing. However they do paywall the best parts of their courses. A work around for this is to simply look at what they want you to learn and look for that topic elsewhere, then come back to Codecademy once you've completed that topic.

08

The #1 Resource I Wish I Had When Learning to Code

Become A Software Engineer



What it's Good for:

- Direct Q&A access to software engineers making $100,000+ in the industry. Step-by-step guidance in what to learn and how to learn on your coding journey. Help crafting your resume. Job & Intern opportunities

How to Use Become a Software Engineer:

- Join in the community for free and ask expert engineers anything you want to know. Follow their courses and guidance all the way to your first $100,000+ coding job.

What's Next?

Now that you know all the free resources I used on my coding journey, you may want private coaching from me so you can become a $100,000+ software engineer faster!

If that's the case, go ahead and schedule a free call with me below to see what the next best steps are for you!

On the call, you and I will come up with a custom plan of action for you so you can see exactly how to become a $100,000+ software engineer using the BreakthroughCoder method.

Click the link below if you are serious about becoming a software engineer.

https://calendly.com/100kcoder/strategy-call-1